

Fundamentals of GPU Programming

Lecture 8

Denis Eremin

- Improvement of an algorithm by changing the control flow (removing thread divergence issues)

Optimization Strategies for the Reduction

We will learn

General Remarks on Reduction

- Common and important data primitive
- Easy to implement in CUDA, hard to get it right
- Many optimization opportunities, good example of fine-tuning techniques

Source

M. Harris, *Optimizing Parallel Reduction with CUDA*

Programming Massively Parallel Processors: A Hands-On Approach

(Hwu, Kirk, Hajj) 4rd Ed.

PROF

Reduction

Reduction uses binary operator on a data set to yield a single result,

e.g.,

or

max

min

addition

multiplication

or a user-defined operator, which is
associative & commutative and
has a well-defined unity value

Reduction, Sequential Implementation

Work complexity $O(n)$

- we want to have same work complexity for the parallel algorithm

(work efficient algorithm)

Parallelization of Reduction

If the operator is commutative and associative,

one can implement a parallel version of the reduction algorithm

- Partition the input data set into chunks, each to be processed by a thread

block

- Use a balanced binary tree reduction algorithm to perform the reduction

inside each of the thread blocks

Parallelization strategy:

Toy (Small Scale) Example

8 input elements, 3

steps (tree levels),

7 operations

—
PROF

Efficiency Analysis

Work complexity

Average parallelism

For average parallelism

Reduction #1: Interleaved Addressing

```
global void reduceO(int *,g_idata, int *g_odata) {  
  
extern shared int sdata[];  
  
}
```

```
// each thread loads one element from global to shared mem
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
sdata[tid] = g_idata[i];
__syncthreads();
```

```
// do reduction in shared mem
for(unsigned int s=1; s < blockDim.x; s *= 2) {
if (tid % (2*s) == 0) {
sdata[tid] += sdata[tid + s];
}
__syncthreads();
}
```

```
II write result for this block to global mem
if (tid == 0) g_odata[blockIdx.x] = sdata[0];
```

Values (shared memory)^10 1 8 -1^0 -2^3 5 -2 ,3^2 7 0 11 0

Step 1 Thread

Stride 1 IDs

Values (^11 1 7) -1 ...:2 -2 8 5 ,5, -3 9 7 11 11 2 2

Step 2 Thread

Stride 2 IDs

Values (^18 1 7) -1 (^6) -2 (^8 5 4) -3 9 7 13 11 2 2

Step 3 Thread

Stride 4 IDs

Values (^24 1 7) -1 (^6) -2 (^8 5 17) -3 9 7 13 11 2 2

Step 4 Thread

Stride 8 IDs

Values (^41 1 7) -1 (^6) -2 (^8 5 17) -3 9 7 13 11 2 2

Divergent warps and slow

% operator are inefficient!

Reduction #2: Interleaved Addressing

Just replace divergent branch in inner loop:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {
```

```
if (tid % (2*s) == 0) {
    sdata[tid] += sdata[tid + s];
```

```
}
```

```
_syncthreads();
```

```
}
```

With strided index and non-divergent branch:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {
```

```
    int index = 2 * s * tid;
```

```
}
```

```
if (index < blockDim.x) {
```

```
    sdata[index] += sdata[index + s];
```

```
}
```

```
_syncthreads();
```

New problem: shared memory conflicts!

Reduction #3: Sequential Addressing

Just replace strided indexing in inner loop:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {
    int index = 2 * s * tid;
```

```
}
```

```
if (index < blockDim.x) {
    sdata[index] += sdata[index + s];
}
_syncthreads();
```

With reversed loop and threadIdx-based indexing:

```
for (unsigned int s=blockDim.x/2; s>0; s>>=1) {
    if (tid < s) {
```

```
}
```

```
sdata[tid] += sdata[tid + s];  
}  
_syncthreads();
```

Values (shared memory) 10 1 8 - 1 0 -2 3 5

Step 1 Thread
Stride 8 IDs

Values -2 -3^2 7 0 11 0

Step 2 Thread
Stride 4 IDs

Values^0 9 3 7 -2 -3 2 7 0 11 0 2

Step 3 Thread
Stride 2: IDs

Values^13 13 0 9 3 7 -2 -3^2 7 0 11 0

Step 4 Thread
Stride 1 IDs

Values^41 20 13 13 0 9 3 7 -2 -3^2 7 0 11 0

Problem: half of the threads are idle on the first iteration!

Reduction #4: First Add During Load

Halve the number of blocks, and replace single load:

If each thread loads one element from global to shared mem

```
unsigned int tid = threadIdx.x;
```

```
unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
sdata[tid] = g_idata[i];
```

```
_syncthreads();
```

With two loads and 'first add of the reduction:

II perform first level of reduction,

II reading from global memory, writing to shared memory

```
unsigned int tid = threadIdx.x;
```

```
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;
```

```
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];
```

```
_syncthreads();
```

1.

When active threads fall inside a single warp,

one can unroll instructions in that warp:

instructions are SIMD synchronous inside a warp

(one no longer needs __syncthreads() and if (s<tid))

Tackling Instruction Overhead Issues

Reduction #5: Unroll 11 the Last Warp

```
device void warpReduce(volatile int* sdata, int tid) {
```

```
sdata[tid] += sdata[tid + 32]; "
```

```
}
```

```
sdata[tid] += sdata[tid + 16];
```

```
sdata[tid] += sdata[tid + 8];
```

```
sdata[tid] += sdata[tid + 4];
```

```
sdata[tid] += sdata[tid + 2];
```

```
sdata[tid] += sdata[tid + 1];
```

```
// later ...
```

IMPORTANT:
For this to be correct,

we must use the

'Volatile' keyword!

```
for (unsigned int s=blockDim.x/2; s>32; s>=>=1) {  
    if (tid < s)  
    }
```

```
sdata[tid] += sdata[tid + s];  
_syncthreads();
```

```
if (tid < 32) warpReduce(sdata, tid);
```

Note: This saves useless work in all warps, not just the last one!

Without unrolling, all warps execute every iteration of the for loop and if statement

2.

Actually, we can completely unroll the for loop using the fact that

number of threads in a thread block is limited and can only be a power of two

number in the reduction algorithm.

To be generic, one can use templates for handling the blocks with unknown size

at the compile time and along with the switch statement to choose during the

run-time execution from different options, pre-defined at the compile time

Specify block size as a template parameter

Statements in red color are evaluated at the compile time

The "switch" statement

Can actually improve it further by using a local register variable instead of sdata[tid]

and storing the result in sdata[tid] afterwards ("cascaded algorithm")!

Persistent Blocks

Number of blocks = #MP x #resident blocks = const for the GPU

Typical results of different optimization strategies (from a similar study)

KERNEL

Neighbored (divergence)

Neighbored (no divergence)

Interleaved

Unroll 8 blocks

Unroll 8 blocks+ last w arp

Unroll 8 blocks + loop + last warp

Templlati zed kernel

KERNELS

reduceGmem

reduceSmem

reduceSmemUnrol 1

red u c eSmemUnrol lDyn

Time (%) Time
2 4 .9.5% 4 .0000us
17.76% 2.84B0us

ELAPSED
TIME (MS)

2.1357

1.1206

0.417 1

0.4169

Call s
1
1

TIME (S)

0.011722

PROF 0.009321

0.006967

0.001422

0.001355

0.001280

0.001253

READ DATA
ELEMENTS

16777216

16777216

16777216

16777216

STEP SPEEDUP CUMULATIVE SPEEDUP

1.26

1.34

4.90

1.05

1.06

1.02

1.26

1.68

8.24

8.65

9.16

9.35

PROF

WRITE DATA TOTAL
ELEMENTS BYTES

131072 67633152

131072 67633152

32768 67239936

32768 67239936

BANDWIDTH
(GB/S)

31.67

60.35

161.21

161.29

```
Avg Min Max Name  
4.0000us 4.0000us 4.0000us reduceSmem()
```

```
2.8480us 2.8480us 2.8480us reduceShfl()
```

Treatment of larger data sets:

- multi-staged reduction kernel calls
- final reduction on CPU
- final reduction on GPU using atomic add function
- final reduction on GPU using threadfencefunction

Final Stage of the Reduction

In a **weakly-ordered** memory model (CUDA)

it is possible that $B = 20$ and $A = 1$

In a **strongly-ordered** memory the only options:

- $B = 20$ and $A = 10$
- $B = 2$ and $A = 1$
- $B = 2$ and $A = 10$

Weakly-Ordered Memory Model of CUDA

Memory Fence Functions

Memory fence functions enforce read/write ordering within different scopes

(but they do not guarantee that all preceding memory functions actually occurred, in contrast to the synchronization barriers!)

There are memory fences on different hierarchy levels: block, grid, host-

devices (`__threadfence_block()`, `__threadfence()`, `__threadfence_system()`)

`__threadfence()` waits until all global and shared memory accesses made by

the calling thread prior to `__threadfence()` are visible to:

- all threads in the thread block for shared memory accesses,

- all threads in the device for global memory accesses

It is not necessary for each of the threads inside a threadblock to reach the memory fence (in contrast to the synchronization barrier)

```
_device_ unsigned int count = 0;
_shared_ bool isLastBlockDone;
_global_ void sum( const float * array, unsigned int N,
floatli!: result)
{
    // Each block sums a subset of the input array
    float partialSum = calculatePartialSum(array, N);
```

```
    if (threadIdx.x == 0) {
```

```
    }
```

```
    // Thread 0 of each block stores the partial sum
    // to global memory
    result[blockIdx.x] = partialSum;
```

```
    // Thread 0 makes sure its result is visible to
    // all other threads
    __syncwritelfence();
```

```
    // Thread 0 of each block signals that it is done
    unsigned int value = atomicInc(&count, gridDim.x);
```

```
    // Thread 0 of each block determines if its block is
    // the last block to be done
    isLastBlockDone = (value == (gridDim.x - 1));
```

```
}
```

```
II Synchronize to make sure that each thread reads  
II the correct value of isLastBlockDone  
_syncthreads();
```

```
if (isLastBlockDone) {
```

```
}
```

```
II The last block sums the partial sums  
II stored in result[0 .. gridDim.x - 1]  
float totalSum = calculateTotalSum(result);
```

```
if (threadIdx.x == 0) {
```

```
}
```

```
II Thread 0 of last block stores total sum  
II to global memory and resets count so that  
II next kernel call works properly  
result[0] = totalSum;  
count = 0;
```

PROF

Homework Assignment 8 (3 points, due on 22.01.26)

For a floating point vector of 40MB size:

- Write a CPU implementation of the reduction algorithm (sum of all the elements). Use it for checking all of GPU outputs!!!
- Implement the reduction algorithm on GPU using the “cascaded algorithm” of Harris with the fixed number of blocks (approximately equal to the number of active blocks on your GPU) and all the optimizations discussed today. Include in kernel the final stage of reduction using the `threadfence` function.

- Compare its performance with the CPU result and with the fastest version of the atomic reduction kernel implemented in the previous homework. Analyze the results.
- Follow the usual homework submission guidelines.